Traffic and Flow Diagram Automation with Claude: Research and Content Plan

Network and Infrastructure Automation · 3 days (8 hours per day) · Intermediate · Compiled 30 October 2026

This document is the working plan for building the course from its outline. It records where the course came from, why the outline is shaped the way it is, what goes into every module (lessons, teaching points, demo, lab, pitfalls and knowledge checks), the lab environment to prepare, and the sources behind each fact. Numbers in square brackets point to the reference list at the end; every URL there was opened and checked against the fact it supports.

- 1. Where this course came from
- 2. How the outline was shaped
- 3. Audience and prerequisites
- 4. Lab environment
- 5. Module-by-module content plan
- 6. Capstone plan
- 7. Promotional video: research behind the script
- References

1. Where this course came from

Client input.

> Client use case 9, "Traffic and Flow Diagram Automation": "Creation for traffic and flow diagram with the help of traceroute and ping access." The client's stated aim for all use cases: "Our Aim is to do automation using Claude for all these content", with course titles of the form "Automation of <> using Claude". Related use cases name the device estate: F5, NetScaler, Radware and Anaya LB load balancers, Cisco ACI, and ServiceNow (SNOW) tickets.

Network and load balancer teams are asked for traffic flow diagrams all the time: for change reviews, audits, incident bridges and application onboarding. In most teams the answer is a manual exercise of running ping and traceroute, reading the output, and drawing boxes in a diagram tool. The result has no link to the evidence, no timestamp, and goes stale the next time a route or pool member changes. Ping and traceroute are also easy to misread: hops that do not answer, routers that rate-limit ICMP, ECMP paths that create false loops, and load balancers that hide what is behind a VIP all show up in real output [3] [4] [14].

The client asked for automation of this flow with Claude, starting from traceroute and ping access. This course treats Claude as a planner, reader and drafter, not as a probe. Deterministic Python runs the probes and returns typed records; read-only adapters add context from F5, NetScaler, Radware Alteon, other load balancers and Cisco ACI; Claude summarizes and drafts a Mermaid diagram where every node cites its evidence; and a named person approves before anything reaches ServiceNow or a shared repository [18] [19] [21].

The result is a pipeline the team can run on every diagram request and on a schedule, with an audit trail a reviewer can follow. Learners leave with code, tests, a guardrail layer and a runbook they can

adapt to their own estate, plus a clear understanding of what a probe-based diagram can and cannot prove.

2. How the outline was shaped

Decision	Rationale
Start with probe limits before any automation.	The client input is 'with the help of traceroute and ping access'. Automating misread output only produces wrong diagrams faster, so Module 1 teaches what ICMP, TTL expiry and probe types actually show, and every later module carries the 'unknown, not guessed' rule [1] [2] [3].
Claude never gets a shell; probes run in deterministic code behind narrow MCP tools.	Probing production networks from an agent with a shell is unsafe and unauditable. Custom tools with fixed schemas, tools=[] in the SDK and a denied Bash tool in Claude Code keep Claude to read-only probes, while hooks enforce scope [15] [18] [20].
Enrich from the load balancers and ACI the client already runs.	The client's other use cases name F5, NetScaler, Radware and Anaya LB, and Cisco ACI. Traceroute stops at a VIP and cannot see pool members or fabric placement, so read-only API adapters fill those gaps with evidence [25] [28] [30] [31].
Mermaid as the diagram source of truth.	Mermaid is text, diffs cleanly in Git, renders natively in GitHub pull requests and can be validated by a command-line renderer, which makes drift detection and review practical. draw.io and Graphviz remain export options [5] [6] [7] [8] [37].
Publish through ServiceNow and GitHub behind a named approval.	Diagram requests arrive as tickets in ServiceNow in the client's environment, and teams keep diagrams in repositories. Writing the result back to the requesting record and to a reviewed pull request fits existing change practice [39] [40] [42].
Add scheduled drift detection with a deterministic diff.	The core complaint is that diagrams go stale. A scheduled job that re-probes within scope and opens a pull request only on a confirmed change keeps diagrams current without noise; Claude explains, code decides [43] [44].

3. Audience and prerequisites

Role	What they need	Starting point
NOC and network operations engineer	Faster, defensible answers to 'what is the path' requests and incident questions.	Uses ping and traceroute daily; little Python or API experience.
Load balancer engineer (F5, NetScaler, Radware)	Show what sits behind VIPs in diagrams without manual screenshots.	Knows tmsh or the NetScaler CLI; some REST exposure.
Data center and ACI engineer	Place endpoints on tenants, EPGs and leaves automatically.	Knows the APIC GUI; limited REST query experience.
Cloud network engineer	Join on-premises paths to VPC or VNet routing evidence.	Familiar with cloud consoles and route tables.
Automation lead	A safe, auditable pattern for using Claude near production networks.	Comfortable with Python, Git and CI; wants governance patterns.

Prerequisites from the outline

- TCP/IP fundamentals and routing basics
- Hands-on familiarity with ping and traceroute
- Introductory Python and Git
- Exposure to at least one load balancer or ACI fabric is helpful, not required

Checks before enrolment

- Can you explain what TTL is and why traceroute increases it per hop?
- Can you read a traceroute output and point to a hop that did not answer?
- Can you write a Python function that runs a command and reads its output?
- Can you clone a Git repository, create a branch, commit and push?
- Do you know what a VIP and a pool member are on at least one load balancer?
- Can you call a REST API with curl and read a JSON response?

Course outcomes

- Explain what ping and traceroute can and cannot prove about a traffic path, and encode those limits in the diagram
- Set up Claude Code, the Claude Agent SDK and an MCP server that exposes read-only probe tools for this workflow
- Build deterministic Python probe runners that return typed path records Claude consumes through tool use and structured JSON outputs
- Enrich hops with F5, Citrix NetScaler, Radware Alteon, other load balancers (Anaya LB) and Cisco APIC context, and render evidence-linked Mermaid diagrams
- Publish diagrams to ServiceNow and GitHub behind scope limits, named approvals and an audit log, and detect path drift on a schedule

Gennoor Tech
For more info - https://gennoor.com

4. Lab environment

Each learner gets a Linux lab VM with Python, Claude Code and the course repository, a small routed lab network with a firewall that drops ICMP, lab load balancers, an APIC (DevNet sandbox or simulator), a ServiceNow developer instance and a GitHub organization. All hostnames and addresses are fictional and no lab component can reach production.

Components

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

Component	Version or edition	Purpose	How to obtain	Notes
Learner workstation	Ubuntu 24.04 LTS VM with Python 3.12, Node.js 20 LTS, Git	Runs Claude Code, the Agent SDK scripts, probe runners and adapters.	Course image provided by the training team.	traceroute, mtr and iputils ping installed; a Windows Server jump host provides tracert captures.
Claude Code and Claude Agent SDK (Python)	Current releases at delivery, pinned in requirements	Workbench, tool use, hooks and structured outputs.	Install per Anthropic documentation with an organization API key or seat.	Settings and hooks shipped in the course repository [15] [18] [20].
Routed lab network	Containerlab or EVE-NG with open-source routers (for example FRRouting)	Produces real traceroute behavior: ECMP, an ICMP-dropping firewall, a rate-limited router.	Built by the lab team from course topology files.	Instructor can shut links to create drift.
F5 BIG-IP Virtual Edition	BIG-IP VE 17.x with LTM	VIP and pool member enrichment.	30-day trial via MyF5, or a lab license [45].	Read-only lab account for enrichment.
NetScaler VPX Express	NetScaler VPX 14.1 Express	lbvserver and service group enrichment.	Free VPX Express download; no license file [46].	No vendor support cases on Express.
Radware Alteon	Alteon VA 32.x or recorded API responses	Alteon enrichment through alteon-sdk.	Customer or partner evaluation; otherwise use recorded responses [30].	Anaya LB and other load balancers are taught through the common adapter using recorded or mocked responses.
Cisco APIC	DevNet ACI Simulator (Always-On or reserved) or ACI Simulator VM	Tenant, EPG, endpoint and L3Out enrichment.	Cisco DevNet sandbox, free with a Cisco account [33].	Always-On is shared; use reserved sandbox or recorded responses for consistent labs.
ServiceNow developer instance	Current family release (personal developer instance)	Request records, Table and Attachment API, Flow Designer.	Free personal developer instance from the ServiceNow developer program.	REST step requires Integration Hub; polling fallback provided [41].
GitHub organization	GitHub Team or Enterprise Cloud	Diagram repository, pull requests, Actions schedule.	Course organization created by the training team.	Self-hosted runner in the lab network for drift jobs.

mermaid-cli	Pinned version in a container image	Deterministic rendering and validation.	npm or the minlag/mermaid-cli image [37].	Same version on laptops and runners.

Fixtures and sample data

Fixture	Contents	Used in
Probe capture pack	Linux and Windows traceroute and ping captures: clean, firewall-dark, rate-limited, ECMP, MPLS-like.	Deterministic probe runners in Python; Path evidence model and structured outputs
Recorded device API responses	F5, NetScaler, Alteon and APIC JSON responses for the lab tenant and VIPs, with fictional addresses.	Enriching hops with load balancer context; Data centre fabric and cloud path context
Scope and budget files	Allowed prefixes, hostnames and probe budgets for the lab.	Guardrails, approvals and audit for probing and publishing
ServiceNow catalog item and flow	Diagram request item, record trigger flow and integration user role.	Publishing to ServiceNow and GitHub
Drift fixture pairs	Before and after path records for real change, ECMP noise and probe failure.	Drift detection and scheduled re-mapping

Lab safety rules

- Labs probe only lab prefixes listed in the scope file; the lab network has no route to production or the internet except for package mirrors.
- Device accounts used by Claude-driven tools are read-only; write accounts exist only for instructors.
- Credentials live in a vault or environment variables and never in prompts, repositories or logs.
- All hostnames, ticket numbers and addresses in materials are fictional.
- Probe budgets are enforced on the lab MCP server to model production behavior.

5. Module-by-module content plan

Day 1

> **Module 1: What a flow diagram must prove, and what probes can tell you**

Goal. Set the evidence standard for a traffic diagram and show exactly which parts of a path ping and traceroute can and cannot prove.

Outline topics.

GenNoor Tech
For more info: https://gennoor.com

- Why hand-drawn traffic diagrams go stale: the request-to-diagram cycle in real NOC and change work
- What ping and traceroute actually show: ICMP and UDP probes, TTL expiry, RTT, per-hop loss, asymmetric return paths
- Where evidence goes dark: firewalls that drop ICMP, load balancers that hide pool members, ECMP and MPLS hops
- The teaching pattern for the course: Claude plans and explains, deterministic code probes, a person approves, everything is logged

Learning objectives.

- Learners can explain how TTL-based traceroute discovers hops and which ICMP messages it relies on.
- Learners can list at least five reasons a hop shows as * or a path looks wrong, including filtering, rate limiting, ECMP and MPLS.
- Learners can distinguish forward-path evidence from return-path assumptions.
- Learners can state the course operating pattern: Claude plans and explains, code probes, a person approves, everything is logged.
- Learners can write a one-page pipeline brief with scope, inputs, outputs and known evidence gaps.

Why diagrams go stale

- Walk the typical request-to-diagram cycle: a ticket asks for 'the flow from branch to the payments VIP', an engineer runs traceroute and ping, draws boxes in a drawing tool, and the result is never re-checked.
- Show the cost: a diagram with no timestamp or probe reference cannot be trusted in a change review or an incident bridge.
- Define the course evidence rule: every node and edge must point to a probe record or a device API record, and anything else is labeled unknown.
- Frame what Claude adds: planning which probes to run, reading raw output, explaining gaps, and drafting the diagram, while deterministic code does the probing.
- Classic traceroute needs about 2n probes for n hops and the path can change during the trace; RFC 1393 tried to fix this with an IP option that never became common, so we live with the limits [1].

How ping and traceroute actually work

- ping sends ICMP Echo (type 8) and expects Echo Reply (type 0); the identifier and sequence fields match replies to requests [2].
- Each traceroute probe has a growing TTL; a router that decrements TTL to zero returns ICMP Time Exceeded (type 11, code 0) [2].
- Linux and Cisco IOS default to UDP probes starting at port 33434; the destination answers with ICMP Port Unreachable (type 3 code 3); Windows tracert uses ICMP Echo instead [3].

- Linux traceroute can also probe with ICMP (-I) or TCP SYN (-T, default port 80), which matters when firewalls filter UDP or ICMP [10].
- Default limits to remember: 30 hops, 3 probes per hop on Linux, a 5 second wait on Linux and 4000 ms on Windows [10] [12].
- Output annotations such as !H, !N, !P and !X tell you the probe was rejected, which is evidence in itself [10].

Where evidence goes dark

- Firewalls that drop ICMP or UDP make traceroute stop at the firewall; the hop after is not missing, it is unseen [10].
- Routers rate-limit ICMP; Cisco limits unreachables to one per 500 ms by default, so a final-hop * can be an artifact [3].
- Loss at an intermediate hop does not mean forwarded traffic is lost; mtr's own documentation warns about this [4].
- ECMP and per-flow load balancing create false loops and diamonds unless the probe header is held constant (Paris traceroute approach) [14].
- MPLS cores may hide hops or show them only if routers append the label stack per RFC 4950 [13].
- Load balancers terminate or proxy connections, so traceroute to a VIP shows the VIP, never the pool members behind it; that context must come from the device API (Module 5).
- Traceroute only shows the forward path from the source; the return path can differ and must be marked as not observed.

The course pattern and the pipeline brief

- Claude plans and explains; Python code runs the probes; device adapters read state; a named person approves publication; every step is logged.
- Claude never receives a shell tool in this course; it gets narrow, read-only tools only (Module 2).
- The pipeline brief records: requester, source, target, protocol and port of interest, allowed prefixes, probe budget, devices to enrich from, and where the result is published.
- The brief also lists known blind spots for the target path, such as firewalls that drop ICMP or an MPLS core.
- The capstone acceptance test is derived from this brief, so it must be specific.

Instructor demo.

1. Run traceroute -n to a lab VIP from the lab client and narrate each line, including a * * * hop behind the lab firewall.
2. Run the same trace with -I and then -T -p 443 and show how the visible hops change with probe type [10].
3. Run tracert /d to the same target from the Windows jump host and compare the output format [12].

4. Run mtr --report for 20 cycles and point out loss at an intermediate hop that does not appear at the destination [4].
5. Trace to the VIP and show that the pool members never appear; open the F5 GUI to show what traceroute cannot see.
6. Write the hop table on a whiteboard with columns: hop, address, RTTs, evidence type, confidence, notes.

Hands-on lab. Trace the path from the lab client to the lab payments VIP by hand, record every hop and every gap, and write the brief for the pipeline you will build.

1. Log in to the lab client and confirm the target VIP and allowed prefix list from the lab handout.
2. Run ping -c 10 -W 2 to the VIP and record loss and average RTT [11].
3. Run traceroute -n -q 3 -w 2 -m 30 to the VIP and save raw output to a text file with a timestamp [10].
4. Repeat with -I and with -T -p 443 and save both outputs.
5. Run mtr --report -c 20 and save the report [4].
6. Fill in the hop table: for each hop note address, RTTs, which probe types answered and a confidence of high, medium or unknown.
7. Mark every gap and write the most likely reason (filtering, rate limiting, ECMP, MPLS, load balancer).
8. Write a one-page pipeline brief using the course template, including scope, budget, enrichment sources and publication targets.

Lab success criteria.

- All raw outputs are saved with timestamps and the command used.
- Every hop in the table has a confidence value and every * has a stated likely reason.
- The brief names allowed prefixes, probe budget, enrichment devices and an approver role.
- The learner can explain why pool members do not appear in the trace.

Deliverable. Manual path record and pipeline brief.

Common pitfalls.

- Treating a * hop as a broken link rather than a router that did not answer.
- Reading intermediate-hop loss in mtr as packet loss on the forwarded path [4].
- Assuming the return path matches the forward path.
- Running traceroute with name resolution on and blaming slow DNS for slow hops; use -n or /d [10] [12].
- Forgetting that Windows and Linux use different probe protocols by default, so results differ through the same firewall [3].

Knowledge check.

1. Which ICMP message does a router send when a traceroute probe's TTL expires? Answer: ICMP Time Exceeded, type 11, code 0 [2].

2. Why might traceroute show a diamond or loop that does not exist? Answer: Per-flow load balancing (ECMP) sends probes with different header fields down different paths; holding the flow fields constant avoids it [14].
3. Why does a trace to a load balancer VIP never show the pool members? Answer: The load balancer owns the VIP and proxies or translates traffic to members, so probes terminate at the VIP; member context must come from the device API.
4. What does a final hop of * * * mean on a Cisco-routed path? Answer: Possibly that ICMP unreachables are rate limited (one per 500 ms by default) or filtered, not that the target is down [3].

Slides and assets to prepare.

- Diagram of TTL expiry with ICMP Time Exceeded and Port Unreachable responses.
- Side-by-side capture of Linux traceroute (UDP, ICMP, TCP) and Windows tracert to the same lab target.
- Slide of the 'where evidence goes dark' list with one lab capture per case.
- Hop table template and pipeline brief template (Markdown).
- One-slide summary of the course operating pattern.

Sources for this module. 1, 2, 3, 4, 10, 11, 12, 13, 14

Module 2: Setting up Claude Code, the Agent SDK and MCP for this workflow

Goal. Build a Claude workbench where Claude can call read-only probe tools and nothing else, with credentials kept out of prompts.

Outline topics.

- Claude Code project layout for the diagram pipeline: CLAUDE.md, allowed tools, repository conventions
- Claude Agent SDK in Python: sessions, custom tool definitions, structured JSON outputs and permission hooks
- An MCP server exposing only read-only probe tools (ping, traceroute, DNS lookup) with scope lists and logging
- Secrets and credentials outside the prompt: environment variables and a vault for device API tokens

Learning objectives.

- Learners can write a CLAUDE.md and a .claude/settings.json that set project conventions and permission rules.
- Learners can define custom tools in the Python Agent SDK and restrict Claude to those tools only.
- Learners can register a stdio MCP server with Claude Code at project scope.
- Learners can add a PreToolUse hook that blocks out-of-scope calls and a PostToolUse hook that logs every call.

- Learners can keep device tokens in environment variables or a vault and out of the model context.

Claude Code project layout

- Repository layout: probes/ (runners), adapters/ (devices), schema/ (Pydantic models), diagrams/ (published .mmd), evidence/ (bundles), mcp_server/, tests/, and CLAUDE.md.
- CLAUDE.md lives at ./CLAUDE.md or ./.claude/CLAUDE.md, is shared through Git, and can be started with /init [16].
- Teach clearly that CLAUDE.md is context, not enforcement; anything that must be blocked goes in permission rules or hooks [16].
- Permission rules are evaluated deny, then ask, then allow, and the first match wins, so a broad deny cannot be undone by a narrow allow [15].
- Put Bash in deny for the probe agent so the tool is removed from Claude's context entirely, and allow only mcp__probe__* [15].
- Remind learners that Bash rules match command text and are not a security boundary; the real control is not giving Claude a shell [15].

Agent SDK sessions and custom tools

- Define tools with the @tool decorator: name, description, input schema and an async handler that returns content [18].
- Wrap tools with create_sdk_mcp_server(name='probe', version='1.0.0', tools=[...]) and pass it in mcp_servers [18].
- Set tools=[] so all built-in tools are removed and Claude can only use your MCP tools; list each tool as mcp__probe__<name> in allowed_tools [18].
- Use a full JSON Schema input with enums and ranges, for example count 1 to 10 and timeout 1 to 5 seconds, so bad arguments fail validation [18].
- Return is_error with a composed message when a probe fails so Claude can react sensibly instead of seeing a raw exception [18].
- Mark probe tools with readOnlyHint, but explain this is metadata only and does not enforce anything [18].

The read-only probe MCP server

- Expose three tools only: ping, traceroute and dns_lookup; each wraps the deterministic runner from Module 3.
- MCP clients discover tools with tools/list and call them with tools/call; results carry content, structuredContent and isError [21].
- The MCP specification says servers MUST validate inputs, implement access controls, rate limit invocations and sanitize outputs; clients SHOULD log tool usage and use timeouts [21].
- Register the server with claude mcp add --scope project probe -- python -m mcp_server; project scope writes .mcp.json and prompts for approval the first time [17].

- Every tool call writes an audit line: timestamp, tool, arguments, caller session, result status and probe ID.

Hooks and secrets

- A PreToolUse hook with HookMatcher(matcher='^mcp__probe__') checks tool_input['target'] against the scope list and returns permissionDecision 'deny' with a reason [20].
- Matchers match tool names only, so argument checks must happen inside the hook body [20].
- A PostToolUse hook appends to the audit log; if hooks disagree, deny wins [20].
- Device API tokens are read by the adapter from environment variables or a vault at call time and never appear in tool arguments, results or prompts.
- Add Read deny rules for .env and secrets/** so Claude Code cannot open them through file tools [15].

Instructor demo.

1. Show the repository tree and the CLAUDE.md with probe conventions and the evidence rule.
2. Open .claude/settings.json with deny: ['Bash'] and allow: ['mcp__probe__*'] and explain evaluation order [15].
3. Run claude mcp add --scope project probe -- python -m mcp_server and show the generated .mcp.json [17].
4. Start an Agent SDK script with tools=[] and the probe server, and ask Claude to trace the lab VIP [18].
5. Ask Claude to run 'ls /' and show it has no tool to do so.
6. Ask Claude to ping 8.8.8.8 and show the PreToolUse hook denying it with a reason [20].
7. Open the audit log and show one line per tool call.

Hands-on lab. Stand up the MCP server, connect it to Claude Code and the Agent SDK, and prove Claude can call a probe tool but cannot run an arbitrary shell command.

1. Clone the course repository and create a Python virtual environment with the pinned requirements.
2. Write CLAUDE.md with the evidence rule, repository layout and test command.
3. Create .claude/settings.json with Bash denied, Read deny rules for secrets, and the probe tools allowed.
4. Implement the MCP server with ping, traceroute and dns_lookup tools calling stub runners.
5. Register the server at project scope and confirm Claude Code lists the tools.
6. Write an Agent SDK script with tools=[], the in-process probe server and a PreToolUse scope hook.
7. Ask Claude to trace an in-scope target, an out-of-scope target and to run a shell command; record what happens.
8. Inspect the audit log for all three attempts.

Lab success criteria.

- Claude successfully calls the in-scope probe and returns its result.
- The out-of-scope probe is denied by the hook with a logged reason.
- No shell command runs, and the transcript shows Claude had no shell tool.
- No credential appears in any prompt, tool argument or log line.

Deliverable. Working Claude workbench with a read-only probe MCP server.

Common pitfalls.

- Relying on CLAUDE.md text to prevent shell use instead of removing the tool [16].
- Adding an allow rule and expecting it to override a broader deny [15].
- Writing a hook matcher that tries to match a target IP; matchers only see tool names [20].
- Forgetting that a project-scoped MCP server needs approval on first use, then assuming it is broken [17].
- Passing device passwords as tool arguments so they end up in transcripts.

Knowledge check.

1. A deny rule says Bash and an allow rule says Bash(traceroute *). Can Claude run traceroute through Bash? Answer: No. Deny is evaluated first and a bare Bash deny removes the tool from context [15].
2. How do you make sure the Agent SDK session has no built-in tools at all? Answer: Pass tools=[] so only your MCP tools remain [18].
3. Where should an argument-level scope check live? Answer: Inside a PreToolUse hook body (and in the tool handler), because matchers only match tool names [20].
4. Name two duties the MCP specification places on servers. Answer: Any two of: validate inputs, implement access controls, rate limit invocations, sanitize outputs [21].

Slides and assets to prepare.

- Starter repository with folder layout, CLAUDE.md template and .claude/settings.json.
- MCP server skeleton (Python) with three tool stubs and audit logging.
- Agent SDK starter script with tools=[] and hook examples.
- Diagram of the trust boundary: Claude, MCP server, runners, devices, vault.
- Cheat sheet of permission rule syntax and evaluation order.

Sources for this module. 15, 16, 17, 18, 20, 21

Module 3: Deterministic probe runners in Python

Goal. Wrap ping and traceroute in tested Python runners that always return a typed record, whatever the network does.

Outline topics.

- Wrapping ping and traceroute with subprocess: timeouts, retries, max hops, packet counts and exit-code handling
- Parsers for Linux and Windows output into a typed path record, with unit tests against saved captures
- Handling the ugly cases: stars, duplicate hops, rate-limited routers and partially answering paths
- Claude requests probes through tool use; code runs them and returns structured results

Learning objectives.

- Learners can call ping and traceroute through subprocess with timeouts, fixed argument lists and exit-code handling.
- Learners can parse Linux and Windows output into one typed hop list.
- Learners can handle stars, duplicate hops, annotations and partial paths without guessing.
- Learners can unit test parsers against saved captures.
- Learners can expose runners to Claude as tools that return structured results.

Running probes safely

- Build the command as a list, never a shell string, and validate the target as an IP or allowed hostname before building it.
- Use subprocess.run(..., timeout=N, capture_output=True); on timeout the child is killed and TimeoutExpired is raised, which the runner turns into a failed record [22].
- ping exit codes: 0 success, 1 no replies (or fewer than count by deadline), 2 other error; record the code rather than treating non-zero as a crash [11].
- Fix probe parameters per run: ping -c 5 -W 2 -w 15; traceroute -n -q 3 -w 2 -m 30 [10] [11].
- Record the tool version (traceroute -V, ping -V) and the exact command in every record.

Parsing Linux and Windows output

- Linux traceroute prints a hop number, then per-probe address and RTT or *; annotations such as !H or !X follow the RTT [10].
- Windows tracert prints three RTT columns then an address; 'Request timed out' rows are all asterisks [12].
- Normalize both into Hop(ttl, responders[], rtts_ms[], annotations[], status) with status of answered, partial or no_reply.
- A hop can have several responders when probes take different ECMP paths; keep them all rather than picking one [14].
- Save each raw capture next to its parsed record so a reviewer can replay the parse.

The ugly cases

- Stars: record no_reply with zero responders; never interpolate an address.
- Duplicate addresses on consecutive hops can mean a router decrementing TTL oddly or a load balancer; flag them, do not merge them.

- Rate-limited routers: with many simultaneous probes (default 16 on Linux) some routers throttle ICMP, so lower -N for consistent output [10].
- Partially answering paths: the destination may answer even when middle hops do not; mark reached_target independently of hop completeness.
- Destination unreachable annotations are evidence of filtering and should be kept as such [10].

Tests and tool exposure

- Build a fixtures folder of captures: clean path, firewall-dark hop, rate-limited final hop, ECMP split, Windows output.
- Write pytest cases asserting hop count, statuses and RTT values for each capture.
- Validate the parsed result with Pydantic model_validate so a parser bug fails loudly with ValidationError [24].
- The MCP tool handler calls the runner and returns the record as JSON content; on failure it returns is_error with a clear message [18].
- Claude requests a probe, code runs it, Claude receives only the structured record and the probe ID.

Instructor demo.

1. Open a raw Linux capture with a * hop and a !X annotation and walk through the parser.
2. Run the runner against a black-hole lab target and show TimeoutExpired being turned into a failed record [22].
3. Show ping returning exit 1 for an unreachable target and the record capturing it [11].
4. Parse a Windows tracert capture into the same record type [12].
5. Ask Claude Code to add a parser test for a new capture, then review the diff line by line before accepting.
6. Call the probe tool from Claude and show the structured JSON that comes back.

Hands-on lab. Implement and test the probe runners with Claude Code, inspect every diff, and run them against three lab targets with different failure modes.

1. Review the provided capture fixtures and expected results.
2. Ask Claude Code to draft the ping runner with timeout and exit-code handling; review and correct the diff.
3. Draft the Linux traceroute parser and run the tests against the fixtures.
4. Add the Windows tracert parser and its tests.
5. Add handling for stars, duplicate hops, annotations and multiple responders.
6. Connect the runners to the MCP tools from Module 2.
7. Run probes against target A (clean), target B (behind a firewall dropping ICMP) and target C (rate-limited).
8. Compare the three records and note what each status means.

Lab success criteria.

Gemnoor Tech
INNOVATE BUILD
For more info https://gemnoor.com

- All parser tests pass, including the ugly cases.
- No runner path uses shell=True or string-built commands.
- A timeout produces a failed record, not an unhandled exception.
- The three lab targets produce three distinct, correctly labeled records.

Deliverable. Tested probe runner library.

Common pitfalls.

- Building commands with f-strings and shell=True, opening an injection path.
- Treating ping exit code 1 as a crash rather than a valid 'no reply' result [11].
- Collapsing multiple responders on one hop into a single address [14].
- Accepting Claude-drafted parsers without running them against real captures.
- Leaving default 16 parallel probes and getting inconsistent results from throttled routers [10].

Knowledge check.

1. What happens to the child process when subprocess.run times out? Answer: It is killed and waited for, then TimeoutExpired is raised [22].
2. Which ping exit code means no replies were received? Answer: 1; 2 means another error [11].
3. What is the default wait per traceroute probe on Windows? Answer: 4000 ms, set with /w [12].
4. Why keep multiple responders for a single TTL? Answer: They show ECMP or load balancing, which a single address would hide [14].

Slides and assets to prepare.

- Capture fixture pack (Linux and Windows) with expected parse results.
- Runner and parser skeleton with pytest harness.
- Slide mapping each output pattern to a hop status.
- Diff review checklist for Claude-drafted code.

Sources for this module. 10, 11, 12, 14, 18, 22, 24

Module 4: Path evidence model and structured outputs

Goal. Define the path record and evidence bundle, and make Claude summarize a path with mandatory probe citations and no invented hops.

Outline topics.

- Pydantic schema for a path: source, target, hops, RTTs, loss, timestamps, probe ID and tool version
- Marking unknown segments explicitly rather than filling gaps; confidence per hop
- Prompting Claude to summarize a path record with mandatory citations to probe IDs
- Evidence bundle layout: raw output, parsed record, Claude summary, all versioned together

Learning objectives.

- Learners can write a Pydantic schema for a path record with explicit unknown segments and per-hop confidence.
- Learners can produce JSON Schema from the model and use it as an Agent SDK output format.
- Learners can prompt Claude to cite probe IDs for every claim and to refuse to fill gaps.
- Learners can lay out a versioned evidence bundle that a reviewer can replay.

The path record schema

- Fields: path_id, source, target, protocol, port, started_at, finished_at, probe_ids[], tool_versions, hops[], reached_target, notes.
- Hop fields: ttl, responders[], rtts_ms[], loss_pct, status (answered, partial, no_reply), confidence (high, medium, unknown), evidence_refs[].
- Unknown segments are first-class objects: from_hop, to_hop, reason (filtered, rate_limited, mpls, lb, unknown), never a guessed address.
- Generate the schema with model_json_schema() and store it in schema/ under version control [23].

Structured outputs from the Agent SDK

- Pass output_format={'type': 'json_schema', 'schema': PathSummary.model_json_schema()} and read ResultMessage.structured_output [19].
- The SDK validates against JSON Schema draft-07; Pydantic targets 2020-12 by default, so keep schemas simple and do not add a newer $schema declaration [19] [23].
- Treat subtype error_max_structured_output_retries, or success without structured_output, as failure [19].
- Re-validate the result with PathSummary.model_validate before saving it [24].

Prompting for cited summaries

- Wrap the path record in XML tags such as <path_record> and give instructions in <instructions> [9].
- Tell Claude what to do: 'cite the probe ID for every hop you mention; for unknown segments state the reason given in the record' [9].
- Give one worked example of a correct summary with citations, since examples steer format reliably [9].
- A deterministic checker rejects any summary that mentions an address not in the record or a probe ID not in probe_ids.
- Summary fields: headline, hop_summaries[{ttl, text, evidence_refs}], gaps[{from_hop, to_hop, reason}], caveats[].

The evidence bundle

- Layout per request: evidence/<request_id>/raw/, parsed/path.json, enrich/, summary.json, diagram.mmd, manifest.json.
- manifest.json lists each file with a SHA-256 hash, tool versions and timestamps.

- The bundle is committed in the same pull request as the diagram so review sees everything together.
- Nothing in the bundle contains credentials; adapters redact tokens before saving API responses.

Instructor demo.

1. Show the Pydantic models and the generated JSON Schema [23].
2. Run the summarizer on a complete path and show structured_output [19].
3. Run it on a path with hops 3 to 5 unknown and show the gaps list instead of invented hops.
4. Edit the prompt to remove the citation instruction and show the checker failing the output.
5. Show the manifest with hashes and explain replay.

Hands-on lab. Define the schema, feed Claude a path with missing hops, and verify the summary cites probe IDs and refuses to invent the gap.

1. Write PathRecord, Hop, UnknownSegment and PathSummary models.
2. Generate JSON Schema and commit it.
3. Write the summarizer agent with output_format set to the PathSummary schema.
4. Write the citation checker that cross-checks addresses and probe IDs.
5. Run the agent on the provided complete path fixture.
6. Run it on the gap fixture and inspect the gaps list.
7. Build the evidence bundle and manifest for one run.
8. Swap in a fixture with a fake address in the summary and confirm the checker rejects it.

Lab success criteria.

- Every hop mentioned in the summary has at least one valid probe ID.
- The gap fixture produces explicit unknown segments with reasons and no invented addresses.
- The checker rejects the tampered summary.
- The bundle manifest lists hashes for every file.

Deliverable. Path evidence schema and summarization agent.

Common pitfalls.

- Using optional fields for unknown hops instead of an explicit unknown segment, which lets gaps disappear silently.
- Trusting structured_output without re-validating or checking citations.
- Declaring a 2020-12 $schema in the output schema and having the run rejected [19].
- Telling Claude only what not to do instead of what to do [9].

Knowledge check.

1. Which result subtype tells you structured output failed after retries? Answer: error_max_structured_output_retries [19].

Gennoor Tech
For more info https://gennoor.com

2. How does the pipeline know Claude did not invent a hop? Answer: A deterministic checker compares every address and probe ID in the summary with the path record.
3. Which Pydantic method generates the JSON Schema? Answer: BaseModel.model_json_schema() [23].

Slides and assets to prepare.

- Pydantic model file and generated schema.
- Prompt template with XML sections and one worked example.
- Citation checker script.
- Evidence bundle layout diagram and sample manifest.

Sources for this module. 9, 19, 23, 24

Day 2

Module 5: Enriching hops with load balancer context

Goal. Use read-only adapters to show what sits behind a VIP on F5, NetScaler, Radware Alteon and other load balancers.

Outline topics.

- F5: virtual servers, pools and pool members through iControl REST and tmsh list ltm virtual
- Citrix NetScaler lbvserver, servicegroup and binding objects through the NITRO API
- Radware Alteon virtual services via Cyber Controller and Alteon REST; a common adapter for other load balancers (Anaya LB)
- Matching a hop or VIP address to a device object so the diagram can show what sits behind it

Learning objectives.

- Learners can query F5 virtual servers and pool members through iControl REST.
- Learners can query NetScaler lbvservers and their service group bindings through NITRO.
- Learners can read Radware Alteon objects from Python and design a common adapter for other load balancers.
- Learners can match a VIP address and port to a device object and attach it to the evidence bundle.

F5 BIG-IP through iControl REST and tmsh

- GET /mgmt/tm/ltm/virtual lists virtual servers; destination holds the address and port and pool names the default pool [25].
- GET /mgmt/tm/ltm/pool/~Common~pool_pay_app/members returns members with address, state, session and monitor [26].
- Use $select=name,destination,pool and expandSubcollections=true to cut calls and payload size [27].

- tmsh list ltm virtual vs_pay_443 is the CLI equivalent learners will know; the REST path mirrors the tmsh tree.
- Use a read-only (Guest or Auditor role) account for enrichment; the adapter never calls POST, PUT, PATCH or DELETE.

Citrix NetScaler through NITRO

- GET /nitro/v1/config/lbvserver returns name, servicetype, ipv46, port, curstate and effectivestate [28].
- Use ?filter=ipv46:10.40.12.25 and attrs= to fetch only matching fields [28].
- GET /nitro/v1/config/lbvserver_servicegroup_binding/<name> returns the bound servicegroupname values [29].
- Authenticate with a session token in Cookie: NITRO_AUTH_TOKEN, or X-NITRO-USER and X-NITRO-PASS headers for single calls [28].

Radware Alteon and the common adapter

- Radware publishes alteon-sdk, a Python API to Alteon over REST, with beans such as SlbNewCfgEnhRealServerTable; minimum supported Alteon versions are 31.0.10.0 and 32.2.2.0 [30].
- The SDK is still a beta release on PyPI, so pin the version and wrap it behind the course adapter interface [30].
- Where Cyber Controller manages the Alteons, confirm with the customer which API is the source of truth before building against it; the course treats it as an optional source.
- Common adapter interface: find_vip(address, port) returns VipRecord(device, vip_name, protocol, port, members[], state, evidence_ref).
- Anaya LB and any other load balancer plug in through the same interface; if no API exists, the adapter returns unknown with a reason.

Matching hops to device objects

- Match on address and port first; a VIP address with several ports needs the port from the brief.
- Partitions and route domains (F5) or traffic domains can make addresses ambiguous; record the partition with the match [25].
- Each adapter call gets an evidence ID (for example F5-77) and its redacted response is saved in enrich/.
- Claude calls f5.get_virtual, netscaler.get_lbvserver and so on as tools; it never receives device credentials.

Instructor demo.

1. Call GET /mgmt/tm/ltm/virtual with $select from curl against the lab F5 and read the JSON [25] [27].
2. Call the pool members endpoint and show a member marked user-down [26].
3. Call NITRO lbvserver with a filter on ipv46 and then the service group binding [28] [29].

4. Read a real server bean from the lab Alteon with alteon-sdk [30].
5. Run the Agent SDK session and ask Claude to explain what sits behind the payments VIP; show the tool calls and evidence IDs.
6. Open the audit log and show each device API call.

Hands-on lab. Build read-only adapters for the lab F5 and NetScaler, let Claude map a VIP to its pool members through tool use, and log every API call.

1. Create read-only lab accounts on the F5 and NetScaler and store tokens in the vault.
2. Implement the F5 adapter: find_vip by destination, then fetch members with expandSubcollections.
3. Implement the NetScaler adapter: lbvserver filter by ipv46, then service group bindings.
4. Write contract tests against recorded API responses for both adapters.
5. Expose both adapters as MCP tools with input validation.
6. Ask Claude to enrich the Module 4 path record and attach results to the evidence bundle.
7. Stretch: implement the Alteon adapter with alteon-sdk against the lab or a recorded response.
8. Review the audit log and confirm only GET calls were made.

Lab success criteria.

- The VIP in the path record is matched to the right virtual server and members on at least two platforms.
- Every enrichment has an evidence ID and a saved redacted response.
- The audit log shows only read calls.
- No credential appears in prompts, tool results or saved responses.

Deliverable. Load balancer enrichment adapters.

Common pitfalls.

- Using an admin account for enrichment because it was already configured.
- Ignoring F5 partitions and matching the wrong VIP [25].
- Pulling every lbvserver field and every object on large appliances instead of filtering [28].
- Building directly against a beta SDK without an adapter layer [30].
- Letting Claude choose arbitrary REST paths instead of fixed adapter tools.

Knowledge check.

1. Which iControl REST parameter returns pool members in the same response as the pool? Answer: expandSubcollections=true [27].
2. Which NITRO object links a NetScaler vserver to a service group? Answer: lbvserver_servicegroup_binding, field servicegroupname [29].
3. What should an adapter return for a load balancer with no usable API? Answer: An unknown result with a reason, so the diagram marks the segment as unknown.

Slides and assets to prepare.

- Adapter interface definition and VipRecord model.
- Recorded API responses for F5, NetScaler and Alteon (fictional addresses).
- Read-only account setup guide for each lab device.
- Slide mapping tmsh commands to iControl REST paths.

Sources for this module. 25, 26, 27, 28, 29, 30

Module 6: Data centre fabric and cloud path context

Goal. Place hops inside the Cisco ACI fabric and the cloud, and decide what stays a labeled unknown.

Outline topics.

- Cisco APIC REST API: tenants, bridge domains, EPGs and L3Outs that a path crosses
- Reading ACI endpoint and interface data to place a hop on a leaf or border leaf
- Hybrid paths: on-premises to cloud VPC or VNet, where traceroute stops and cloud route tables begin
- Deciding what to enrich and what to leave as a labelled unknown

Learning objectives.

- Learners can authenticate to APIC and run class and managed object queries with filters.
- Learners can find the tenant, EPG and leaf where an endpoint IP lives.
- Learners can add cloud route table or next hop evidence where traceroute stops.
- Learners can justify which segments to enrich and which to leave as unknown.

APIC REST basics

- Log in with POST /api/aaaLogin.json and an aaaUser name and password; reuse the APIC-cookie token [31].
- Class query: /api/class/fvTenant.json; MO query: /api/mo/uni/tn-pay.json [31].
- query-target scopes to self, children or subtree; query-target-filter takes eq(fvCEp.ip,"10.40.30.11"); rsp-subtree is no, children or full [31].
- Use a read-only APIC user with a security domain limited to the tenants in scope.

Locating an endpoint

- fvCEp represents a learned endpoint; its dn contains tenant, application profile and EPG [32].
- The child fvRsCEpToPathEp holds the leaf and interface path, so request children with rsp-subtree=children and rsp-subtree-class [32].
- Turn the path dn (for example topology/pod-1/paths-103/pathep-[eth1/12]) into leaf-103, eth1/12 for the diagram.
- L3Outs and border leaves are where traffic leaves the fabric; record the L3Out name as the exit point.
- Endpoint data is learned state and can age out; record the query time with every result.

Hybrid and cloud paths

- Traceroute into a cloud network often stops at the gateway or shows provider hops only; the route table is better evidence.
- AWS route tables map a destination CIDR to a target such as a VPN connection, NAT gateway or peering connection [34].
- Azure Network Watcher next hop returns next hop type, IP and route table ID for a destination [35].
- Cloud lookups use read-only roles and are tools like any other, logged with evidence IDs.
- If no cloud access is granted, the diagram shows a labeled cloud boundary with 'route table not inspected'.

What to enrich and what to leave unknown

- Enrich when a read-only API can answer with a timestamped fact; leave unknown when the only source would be a guess or a stale document.
- Never enrich from memory or from Claude's general knowledge of typical designs.
- Write the enrichment decision into the evidence bundle so the reviewer can see why a segment is unknown.
- Agree the enrichment policy with the approver role in advance.

Instructor demo.

1. Log in to the DevNet ACI sandbox or lab APIC with curl and save the token [31] [33].
2. Run a class query for fvTenant and an MO query for one tenant [31].
3. Find a lab endpoint with query-target-filter on fvCEp.ip and include fvRsCEpToPathEp children [32].
4. Show an AWS route table and an Azure next hop result for the cloud segment of the lab path [34] [35].
5. Show the enriched evidence bundle with fabric and cloud records and one deliberately unknown segment.

Hands-on lab. Query the lab APIC for the EPGs and L3Out a path traverses and attach fabric context to the evidence bundle.

1. Create the APIC adapter with login, class query and MO query functions.
2. Write apic.get_endpoint(ip) returning tenant, AP, EPG, leaf and interface.
3. Write apic.get_l3out(tenant) returning L3Out names and border leaf nodes.
4. Test both against recorded APIC responses.
5. Expose the functions as MCP tools and run Claude enrichment on the Module 5 bundle.
6. Add a cloud segment from a recorded route table or next hop response, or label it unknown.
7. Record the enrichment decisions in the bundle.

Lab success criteria.

- Each pool member IP is placed on a tenant, EPG and leaf with an evidence ID.

- The exit L3Out is recorded for the outbound segment.
- The cloud segment is either evidenced or explicitly unknown with a reason.
- All APIC calls in the audit log are read-only.

Deliverable. Fabric-aware path record.

Common pitfalls.

- Querying fvCEp without children and wondering where the leaf is [32].
- Using the APIC admin account for read-only enrichment.
- Treating endpoint learning data as permanent configuration.
- Filling the cloud segment from an old design document instead of evidence.

Knowledge check.

1. Which child object of fvCEp tells you where the endpoint attaches to the fabric? Answer: fvRsCEpToPathEp [32].
2. What does rsp-subtree default to in an APIC query? Answer: no, so children are not returned unless requested [31].
3. What three facts does Azure next hop return? Answer: Next hop type, IP address and route table ID [35].

Slides and assets to prepare.

- APIC object model slide: tenant, AP, EPG, BD, L3Out, fvCEp.
- Recorded APIC responses for the lab tenant.
- Cloud evidence examples for AWS and Azure (fictional).
- Enrichment decision checklist.

Sources for this module. 31, 32, 33, 34, 35

Module 7: Generating the traffic and flow diagram with Mermaid

Goal. Have Claude draft Mermaid from the evidence bundle and use deterministic tools to validate, render and check every node against evidence.

Outline topics.

- Mermaid flowchart and sequence syntax for hop-by-hop traffic and request-response flow
- Claude drafts the diagram from the evidence bundle; a deterministic renderer validates the syntax
- Every node carries the probe ID that placed it; unknown segments are styled as unknown
- Diagram conventions: VIP, pool, firewall, fabric and cloud boundary shapes and a legend

Learning objectives.

- Learners can write Mermaid flowcharts and sequence diagrams for network paths.
- Learners can prompt Claude to draft diagram code that carries evidence IDs on every node.

Gennoor Tech TRAINING INNOVATE BUILD for more info - https://gennoor.com

- Learners can validate and render Mermaid with mermaid-cli and reject unsupported nodes.
- Learners can apply course diagram conventions and a legend.

Mermaid for traffic paths

- Use flowchart LR for hop-by-hop paths; subgraph blocks for site, fabric and cloud boundaries [5].
- Use dotted links (-.-) for unknown segments and thick links (===) for the primary path [5].
- Shapes can be set with the A@{ shape: ... } syntax (Mermaid 11.3.0 and later); check GitHub's Mermaid version with an info block before relying on it [5] [6].
- classDef unknown fill:#eee,stroke-dasharray: 5\,5 then class hop3 unknown; escape commas in dasharray [5].
- Avoid a lowercase end inside labels; it breaks flowcharts [5].
- Use sequenceDiagram for request-response flows such as client to VIP to member and back [38].

Claude drafts, code validates

- Claude receives the bundle summary and must output Mermaid only, inside a fixed structured output field [9] [19].
- A generator checker parses node IDs and confirms each maps to a hop, VIP, member or unknown segment in the bundle.
- mmdc -i diagram.mmd -o diagram.svg renders the file; a parse failure stops the pipeline [37].
- mermaid.parse() can validate without rendering in a Node step; keep securityLevel strict so labels cannot inject HTML [36].
- Rendering happens in the pipeline container (minlag/mermaid-cli) so every run uses the same version [37].

Evidence on every node

- Node labels carry a short evidence line, for example lb-f5-01 vs_pay_443 [F5-77].
- Unknown nodes say why: 'hop 3 unknown: no reply [P-0218]'.
- Edges carry measured RTT where available and nothing where not measured.
- A legend subgraph explains shapes, line styles and evidence prefixes.

Other formats

- GitHub renders mermaid fenced blocks in pull requests and Markdown, so reviewers see the diagram in place [6].
- Teams on draw.io can import the same node and edge list as CSV with # connect and # identity lines [7].
- Graphviz DOT with cluster subgraphs is a fallback for large paths [8].
- Keep Mermaid as the source of truth and generate other formats from the same evidence.

Instructor demo.

1. Show a hand-written Mermaid flowchart of the lab path with subgraphs and a dotted unknown segment [5].
2. Run the Claude diagram agent on the enriched bundle and show the Mermaid it returns.
3. Render with mmdc in the container and open the SVG [37].
4. Remove the evidence ID from one node and show the checker rejecting it.
5. Add a node with lowercase end in the label and show the render failure [5].
6. Push to a branch and show the diagram rendering in a GitHub pull request [6].

Hands-on lab. Generate Mermaid for the lab path, render it, break the evidence on purpose and watch the renderer reject an unsupported node.

1. Write the diagram conventions file: shapes, classDef styles and legend.
2. Write the diagram agent prompt with one worked example.
3. Generate Mermaid for the lab bundle.
4. Run the evidence checker and fix any issues in the prompt, not by hand-editing output.
5. Render with mmdc to SVG and PNG.
6. Add a fake node with no evidence and confirm the checker rejects it.
7. Introduce a syntax error and confirm rendering fails and the pipeline stops.
8. Optional: export the same graph as draw.io CSV.

Lab success criteria.

- The rendered diagram shows every hop, VIP, member and unknown segment from the bundle.
- Every node has an evidence ID that exists in the bundle.
- Unknown segments use the unknown style and say why.
- Both broken cases stop the pipeline with a clear error.

Deliverable. Evidence-linked Mermaid diagram generator.

Common pitfalls.

- Using shape syntax newer than the Mermaid version GitHub runs [6].
- Hand-editing Claude's Mermaid and losing reproducibility.
- Rendering on laptops with different mermaid-cli versions.
- Unescaped commas in style definitions [5].

Knowledge check.

1. How do you find which Mermaid version GitHub uses? Answer: Put info in a mermaid code block [6].
2. Which command renders a Mermaid file to SVG? Answer: mmdc -i input.mmd -o output.svg [37].
3. What does securityLevel strict do? Answer: Encodes HTML tags in text and disables click functionality [36].

GenNoorTech INNOVATE. BUILD. For more info - https://gennoor.com

Slides and assets to prepare.

- Diagram conventions sheet with shapes, styles and legend.
- Prompt template and worked example for the diagram agent.
- Container image definition with pinned mermaid-cli.
- Before and after diagrams: hand-drawn versus evidence-linked.

Sources for this module. 5, 6, 7, 8, 9, 19, 36, 37, 38

Module 8: Guardrails, approvals and audit for probing and publishing

Goal. Enforce scope, rate limits, approvals and audit in code so the pipeline is safe on production networks.

Outline topics.

- Scope lists: which prefixes and hostnames may be probed, enforced in code not in the prompt
- Rate limits and probe budgets so automated tracing never becomes a ping storm
- Named approver required before a diagram is written to the CMDB or merged into a shared repository
- Audit log of every tool call, device API query, model output and approval decision

Learning objectives.

- Learners can enforce a prefix and hostname scope list in code at more than one layer.
- Learners can implement per-target and per-hour probe budgets.
- Learners can require a named approver before any write to ServiceNow or a shared repository.
- Learners can produce an audit log a reviewer can follow end to end.

Scope lists in code

- Scope file lists allowed prefixes and hostnames; resolve hostnames first and check every resolved address.
- Enforce in three places: the PreToolUse hook, the MCP tool handler and the runner itself [20] [21].
- The prompt may mention scope for clarity, but scope is never enforced by the prompt.
- Out-of-scope requests return a denial with a reason and are logged [20].

Rate limits and probe budgets

- The MCP specification requires servers to rate limit tool invocations [21].
- Budget example: 50 probes per hour per pipeline, 3 per target per run, at most 2 concurrent runs.
- Lower traceroute parallelism (-N) to avoid triggering ICMP throttling on routers [10].
- When the budget is exhausted, the tool returns is_error with 'budget exhausted' and Claude must report it [18].

Gennoor Tech TRAINING INNOVATE BUILD For more info - https://gennoor.com

Approval gate

- Publishing tools are separate from probing tools and are not given to the drafting agent.
- The publish step checks the approval record: approver name, role, time, request ID and the hash of the bundle approved.
- If the bundle hash changes after approval, the approval is void.
- MCP clients should present confirmation for sensitive operations and keep a human able to deny invocations [21].

Audit log

- Log every tool call, device query, model output hash and approval decision as JSON lines.
- PostToolUse hooks write the tool call side; adapters write the device side [20].
- Store logs append-only and outside the repository; ship to the SIEM if the customer has one.
- The audit export for a request is attached to the change record in Module 9.

Instructor demo.

1. Show the scope file and the three enforcement points.
2. Ask Claude to probe 10.99.7.9 and show the hook denial and log line [20].
3. Exhaust the probe budget in a loop and show the error Claude receives [21].
4. Attempt to publish without an approval record and show the refusal.
5. Approve, then alter the bundle, and show the approval becoming void.
6. Export the audit log for one request.

Hands-on lab. Add scope enforcement, rate limits and an approval gate, then attempt an out-of-scope probe and an unapproved publish and show both are refused and logged.

1. Write the scope file and loader with hostname resolution.
2. Add scope checks to the hook, tool handler and runner.
3. Add a token-bucket budget to the MCP server.
4. Write the approval record model and a CLI for an approver to sign a bundle hash.
5. Gate the publish function on a valid approval.
6. Try an out-of-scope probe, exceed the budget and attempt an unapproved publish.
7. Export the audit log and walk through it with a partner.

Lab success criteria.

- The out-of-scope probe is refused at the hook and would be refused at the handler if the hook were removed.
- Budget exhaustion is reported cleanly.
- Unapproved and stale-approval publishes are refused.
- The audit log tells the full story without reading code.

Deliverable. Guardrail and audit layer.

Common pitfalls.

- Checking scope on hostnames but not on resolved addresses.
- Enforcing scope only in the prompt or CLAUDE.md [16].
- Giving the drafting agent the publish tool 'for convenience'.
- Approvals that are not tied to a bundle hash.

Knowledge check.

1. Why enforce scope in the runner as well as the hook? Answer: Defense in depth: hooks can be misconfigured or bypassed by another caller, and the runner is the last point before packets leave.
2. What makes an approval void in this design? Answer: Any change to the bundle hash after approval.
3. Which MCP security duty covers probe budgets? Answer: Servers must rate limit tool invocations [21].

Slides and assets to prepare.

- Scope file example and enforcement diagram.
- Approval record schema and approver CLI.
- Audit log sample for one request.
- Slide of the 'ping storm' risk with budget math.

Sources for this module. 10, 16, 18, 20, 21

Day 3

Module 9: Publishing to ServiceNow and GitHub

Goal. Take a ServiceNow request in and publish an approved diagram to the record and to a GitHub pull request.

Outline topics.

- ServiceNow Table API to update the incident, change or CI record that requested the diagram, and the Attachment API to attach the diagram and evidence bundle to it
- Flow Designer trigger: a request record starts the pipeline and receives the result
- GitHub pull request with the Mermaid file, evidence bundle and Claude-written summary; a person reviews and merges
- Repository layout for diagrams per application and per site

Learning objectives.

- Learners can read and update a ServiceNow record through the Table API.
- Learners can attach the diagram and evidence bundle through the Attachment API.
- Learners can trigger the pipeline from Flow Designer and return the result.

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info: https://gennoor.com

- Learners can open a GitHub pull request with the diagram, bundle and Claude summary.

ServiceNow Table API

- Read the request: GET /api/now/table/sc_req_item?sysparm_query=number=RITM0041877&sysparm_fields=sys_id,short_description [39].
- Update with PATCH /api/now/table/{table}/{sys_id} to write work notes and status [39].
- Use sysparm_fields to keep responses small and sysparm_limit on list calls [39].
- The integration user has a role limited to the tables it needs; CMDB writes need an extra, approved role.

Attachment API

- POST /api/now/attachment/file?table_name=sc_req_item&table_sys_id=<sys_id>&file_name=blr-dc1.svg with the binary body [40].
- Attach the SVG, the Mermaid source, the summary and a zipped evidence bundle.
- Use POST /api/now/attachment/upload when you need multipart form data [40].
- Record each attachment sys_id in the audit log.

Flow Designer trigger

- A catalog item or record trigger starts a flow when a diagram request is submitted.
- The flow uses a REST step to call the pipeline endpoint; the REST step needs Integration Hub and supports connection aliases and MID Server execution [41].
- The pipeline responds asynchronously by updating the record when done.
- If Integration Hub is not licensed, fall back to the pipeline polling the Table API on a schedule.

GitHub pull request

- Branch name diagram/<request>; commit diagrams/<app>/<site>.mmd and evidence/<request>/ [6].
- Open the pull request with POST /repos/{owner}/{repo}/pulls with title, head, base and body containing Claude's summary [42].
- CODEOWNERS for diagrams/<app>/ requests the right reviewers; the merge is the approval for the repository copy.
- Repository layout per application and per site keeps history readable.

Instructor demo.

1. Submit a diagram request in the lab ServiceNow instance.
2. Show the flow firing and the REST step calling the pipeline [41].
3. Show the pipeline reading the record through the Table API [39].
4. After approval, show attachments arriving on the record [40].
5. Show the pull request with the rendered diagram and summary [42] [6].
6. Merge and show the record updated with the merge link.

Hands-on lab. Run the pipeline from a ServiceNow request to a GitHub pull request and complete the approval end to end.

1. Create the integration user and role in the lab ServiceNow instance.
2. Write the ServiceNow adapter: get request, patch work notes, attach files.
3. Build the flow with a record trigger and REST step, or configure polling.
4. Write the GitHub publisher that creates a branch, commits files and opens a pull request.
5. Submit a request and let the pipeline run to the approval gate.
6. Approve as the named approver and watch publication.
7. Review and merge the pull request; confirm the record shows the link.

Lab success criteria.

- The request record shows the diagram, bundle and summary as attachments.
- The pull request contains the Mermaid file, bundle and summary.
- Nothing was published before the approval record existed.
- The audit log links request, approval, attachments and pull request.

Deliverable. Request-to-publication workflow.

Common pitfalls.

- Giving the integration user admin rights.
- Using PUT instead of PATCH and clearing fields on the record [39].
- Forgetting table_sys_id and attaching to nothing [40].
- Assuming the REST step is available without Integration Hub [41].

Knowledge check.

1. Which three query parameters does POST /now/attachment/file require? Answer: table_name, table_sys_id and file_name [40].
2. Which fields must a create pull request call include? Answer: head and base, plus title (or an issue) [42].
3. What is needed to call the pipeline from a flow REST step? Answer: An Integration Hub subscription and a connection, optionally through a MID Server [41].

Slides and assets to prepare.

- ServiceNow catalog item and flow export for the lab.
- Adapter code for Table and Attachment APIs.
- GitHub publisher script and PR body template.
- Sequence diagram of request to publication.

Sources for this module. 6, 39, 40, 41, 42

Gennoor Tech INNOVATE BUILD For more info https://gennoor.com

Module 10: Drift detection and scheduled re-mapping

Goal. Re-run in-scope probes on a schedule, detect real path changes deterministically, and raise a pull request with Claude's explanation only when something changed.

Outline topics.

- GitHub Actions schedule that re-runs probes within scope and compares against the published path
- Deterministic diff of path records; Claude explains the change in plain language
- Opening a pull request only when a path changed, with the diff and evidence attached
- Operator runbook: false drift from ECMP, maintenance windows and probe failures

Learning objectives.

- Learners can write a scheduled GitHub Actions workflow and understand its timing limits.
- Learners can diff two path records deterministically and classify changes.
- Learners can have Claude explain a diff in plain language with structured output.
- Learners can write an operator runbook for false drift.

Scheduling the job

- on: schedule with cron '21 21 * * *' runs at 21:21 UTC (02:51 IST); cron is UTC unless a timezone is set [43].
- Runs use the default branch, the minimum interval is 5 minutes, and runs can be delayed or dropped at busy times such as the top of the hour, so avoid minute 0 [43].
- Public repositories disable scheduled workflows after 60 days without activity [43].
- The runner must sit inside the network that can reach the targets, so use a self-hosted runner in the lab network.
- Probes still pass through the same scope list and budget as the interactive pipeline.

Deterministic diff

- Compare hop sequences by responder sets, not single addresses, to avoid ECMP noise [14].
- Classify: new hop, removed hop, changed responder, newly unknown, newly answering, VIP member change.
- Ignore RTT changes below a threshold; RTT is not path.
- Require two consecutive runs to agree before raising drift, to filter one-off probe failures.

Claude explains the change

- Feed the diff and both records to Claude with a structured output schema: summary, changes[{type, hops, evidence_refs}], likely_causes[], confidence [19].
- Claude must cite probe IDs for both the old and new state.
- The diff decides whether a PR is opened; Claude only explains.
- The PR body contains the diff, Claude's explanation and links to the bundles [42].

Pull requests from automation and the runbook

- A PR opened with GITHUB_TOKEN creates pull_request runs in an approval-required state, so a person must approve CI on drift PRs [44].
- Grant only contents: write and pull-requests: write to the job.
- Runbook: ECMP flapping, maintenance windows, probe failures, firewall changes that hide hops, and how to suppress a known false drift.
- Close stale drift PRs automatically if the path returns to the published state.

Instructor demo.

1. Show the scheduled workflow file and explain the cron expression and timezone [43].
2. Trigger the job with workflow_dispatch to avoid waiting.
3. Change the lab path by shutting a link and show the second run confirming drift.
4. Show the deterministic diff and Claude's structured explanation [19].
5. Show the drift PR and the approval-required CI banner [44].
6. Restore the link and show the stale PR closed.

Hands-on lab. Schedule the re-mapping job, change the lab path, and review the drift pull request Claude explains.

1. Register the self-hosted runner in the lab network.
2. Write the workflow with schedule and workflow_dispatch triggers.
3. Implement the diff and classification with tests on fixture pairs.
4. Add the explanation agent with structured output.
5. Add the PR opener that runs only on confirmed drift.
6. Change the lab path and run the job twice.
7. Review the PR and approve its CI run.
8. Write the runbook section for one false-drift case you observed.

Lab success criteria.

- No PR is opened when the path is unchanged.
- A confirmed change produces one PR with diff, explanation and evidence links.
- ECMP-only variation does not raise drift.
- The runbook covers at least three false-drift causes.

Deliverable. Drift detection job and runbook.

Common pitfalls.

- Scheduling at minute 0 and seeing delayed or dropped runs [43].
- Running the schedule on GitHub-hosted runners that cannot reach internal targets.
- Letting Claude decide whether drift happened.
- Expecting CI to run automatically on PRs opened with GITHUB_TOKEN [44].

Genneor Tech TRAINING INNOVATE BUILD for more info - https://genneor.com

Knowledge check.

1. In what timezone does a GitHub Actions cron run by default? Answer: UTC [43].
2. Why compare responder sets instead of single addresses? Answer: ECMP can produce different responders per run without a real path change [14].
3. What happens to CI when the drift job opens a PR with GITHUB_TOKEN? Answer: The pull_request run waits for approval by a user with write access [44].

Slides and assets to prepare.

- Workflow file template.
- Diff classification table and fixture pairs.
- Drift PR template.
- Operator runbook template.

Sources for this module. 14, 19, 42, 43, 44

Module 11: Capstone: request to approved flow diagram, end to end

Goal. Run the full pipeline from a ServiceNow request to an approved, published and monitored diagram, and defend it to the group.

Outline topics.

- A ServiceNow request in, probes run, hops enriched across F5, NetScaler, Radware, other load balancers (Anaya LB) and ACI, Mermaid diagram out
- No publication without a named approver; audit trail reviewed
- Acceptance criteria, limitations statement and team demo

Learning objectives.

- Learners can run the full pipeline across probes, enrichment, diagram, approval and publication.
- Learners can show the audit trail for one request from start to finish.
- Learners can state acceptance criteria and a limitations statement for their diagram.
- Learners can demo the result and answer reviewer questions with evidence.

Capstone scenario

- A ServiceNow request asks for the payments flow from a branch client to the payments VIP at the lab data center.
- The path crosses a firewall that drops ICMP, an F5 or NetScaler VIP, an Alteon or Anaya LB segment through the common adapter, and the ACI fabric.
- Teams must produce the diagram, bundle, PR, ServiceNow update and drift job.
- The instructor injects one surprise: a pool member change or a new hop mid-exercise.

Acceptance and limitations

- Acceptance criteria come from the Module 1 brief and the guardrail rules.
- The limitations statement lists each unknown segment and why it is unknown.
- Return path is stated as not observed unless separately probed from the far side.
- Every claim in the demo must be backed by an evidence ID.

Review and demo

- Peer review uses the audit log, not the code.
- Demo: request in, probes, enrichment, diagram, approval, publication, drift check.
- Reviewers try one out-of-scope probe and one unapproved publish.
- Teams close with what they would change before production.

Instructor demo.

1. Brief the scenario and show the acceptance checklist.
2. Show a reference run end to end in under ten minutes.
3. Inject the surprise change after teams publish their first version.
4. Model a good review by asking for evidence behind two nodes.

Hands-on lab. Capstone build and demo.

1. Submit the capstone request in ServiceNow.
2. Run probes and enrichment for all segments.
3. Generate, check and render the diagram.
4. Obtain approval from the designated approver.
5. Publish to ServiceNow and GitHub.
6. Run the drift job after the injected change and handle the PR.
7. Present the demo and limitations statement.

Lab success criteria.

- Every node in the final diagram has valid evidence.
- No publish occurred without approval, shown in the audit log.
- The injected change is detected and explained.
- The limitations statement is accurate and complete.

Deliverable. End-to-end traffic and flow diagram pipeline.

Common pitfalls.

- Skipping the limitations statement because the diagram 'looks complete'.
- Manually fixing a node in the diagram to save time.
- Approving your own team's publish.

Knowledge check.

1. What must every node in the capstone diagram have? Answer: An evidence ID that exists in the bundle.

Gennoor Tech
TRAINING INNOVATE BUILD
For more info - https://gennoor.com

2. How do you show a reviewer nothing was published without approval? Answer: The audit log shows the approval record, its bundle hash and the publish events after it.
3. How is the return path represented? Answer: As not observed, unless probed separately from the far side.

Slides and assets to prepare.

- Capstone brief and acceptance checklist.
- Reference run recording.
- Peer review form.
- Limitations statement template.

Sources for this module. 14, 20, 39, 40, 42

6. Capstone plan

A repeatable pipeline that takes a flow diagram request from ServiceNow, runs ping and traceroute through deterministic code, enriches every hop from load balancer and ACI APIs, has Claude draft an evidence-linked Mermaid diagram, and publishes it to ServiceNow and GitHub only after a named approval, with drift detection on a schedule.

Teams build a repeatable pipeline that takes a flow diagram request from ServiceNow, runs ping and traceroute through deterministic code, enriches hops from load balancer and ACI APIs, has Claude draft an evidence-linked Mermaid diagram, and publishes to ServiceNow and GitHub only after a named approval, with drift detection on a schedule.

Milestones

1. Request received and brief confirmed from the ServiceNow record.
2. Probes run within scope and budget; path record saved.
3. Enrichment complete across load balancers and ACI, with unknowns labeled.
4. Diagram drafted, checked and rendered.
5. Approval recorded against the bundle hash.
6. Published to ServiceNow and GitHub; PR merged.
7. Drift job detects and explains the injected change.

Evaluation notes

- Score evidence integrity first: any node without valid evidence fails the capstone.
- Check the audit log for approval-before-publish and refused out-of-scope probes.
- Assess the limitations statement for accuracy, including the return path.
- Assess clarity of Claude's summary and drift explanation for a non-specialist approver.
- Assess code quality: tests on parsers and adapters, no shell=True, credentials handled correctly.

Assessment weights

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

7. Promotional video: research behind the script

Opening scenario. The opening scenario (a ticket asks for the flow to a payments VIP, an engineer runs traceroute and ping and draws by hand, and the path changes) is a teaching scenario, but it is grounded in documented behavior: traceroute paths can change during and between traces, and load balancing creates path artifacts [1] [14].

GenNoorTech TRAIN. INNOVATE. LEAD. For more info - https://gennoor.com

Claim or on-screen text	Basis
"What if Claude ran the probes, read every hop, and drew it for you?"	Course framing. In the design, deterministic code runs the probes and Claude requests them through tool use and reads structured results; the narration's 'ran the probes through code you trust' is the accurate version [18] [21].
11 build modules, 3 days, hands-on	Matches content.py: 11 modules over 3 days.
4 device families enriched	F5, NetScaler, Radware Alteon and Cisco ACI, each with a documented read API [25] [28] [30] [31]. Anaya LB is covered through the common adapter; no public Anaya LB API documentation was found, so it is taught with mocked or customer-provided interfaces.
0 hops drawn without evidence	A design rule enforced by the evidence checker in the diagram module, not an external fact.
Terminal: traceroute 10.40.12.25 max-hops 30 timeout 2s; hop 3 * * *; record marks hop 3 UNKNOWN	Realistic: 30 is the default maximum hops and * marks no reply within the wait time [10]. The addresses, record ID P-0218 and the flowmap command are fictional teaching examples.
f5.get_virtual returns vs_pay_443 → pool_pay_app with members	Consistent with iControl REST virtual (destination, pool) and pool members resources [25] [26]; names and addresses are fictional.
apic.get_epg places 10.40.30.11 in tenant pay, EPG app-web, leaf-103	Consistent with fvCEp and fvRsCEpToPathEp lookups through APIC REST [31] [32]; the values are fictional.
mermaid render: OK lint: OK	Rendering and parsing with mermaid-cli and mermaid.parse are real validation steps [36] [37]; 'lint' here means the course evidence checker.
Probe REFUSED outside scope list; publish REFUSED no approver	Design behavior implemented with PreToolUse hooks and the approval gate [20] [21].
"Rate limits and probe budgets keep automated tracing from becoming a ping storm"	The MCP specification requires servers to rate limit tool invocations, and traceroute documentation notes routers throttle ICMP under many parallel probes [21] [10].
Result written to the ServiceNow record through the Table API, diagram attached through the Attachment API, Flow Designer starting the pipeline	Table API PATCH, Attachment API POST /now/attachment/file and the Workflow Studio REST step are documented; the REST step needs Integration Hub, which the promo does not mention [39] [40] [41].
"cmdb_ci_service updated" (cards panel)	Teaching scenario. The course writes to the requesting record by default; CMDB writes require an extra approved role and are optional. No CMDB-specific source was cited.

GitHub Actions schedule re-runs probes and opens a pull request only when something changed; drift job nightly, next: 02:30 IST	Scheduled workflows use UTC cron and can be delayed at busy times; the course recommends an off-the-hour minute, so the 02:30 IST time on screen is illustrative [43].
"Claude never guesses a path. If a hop did not answer, the diagram says so."	Design rule enforced by the schema's explicit unknown segments and the citation checker; supported by documented reasons hops do not answer [3] [4] [10].
Labs use lab load balancers, a lab APIC and fictional hostnames	Lab plan uses BIG-IP VE trial, NetScaler VPX Express and DevNet ACI sandboxes [45] [46] [33].

References

[1] RFC 1393: Traceroute Using an IP Option. IETF / RFC Editor. https://www.rfc-editor.org/rfc/rfc1393
Supports: Module 1: why classic traceroute needs many probes and can see a changing path. Classic TTL-based traceroute generates about 2n packets for n hops; RFC 1393 proposed an IP Traceroute option and ICMP Traceroute message (type 30) so the path is logged with n+1 packets and does not suffer from a changing path. It is Experimental and obsoleted by RFC 6814 (the RFC Editor header now shows it as Historic).

[2] RFC 792: Internet Control Message Protocol. IETF / RFC Editor. https://www.rfc-editor.org/rfc/rfc792
Supports: Module 1: ICMP Echo, Echo Reply and Time Exceeded. ICMP Echo is type 8 and Echo Reply type 0, with identifier and sequence number fields to match replies to requests and the data echoed back unchanged; Time Exceeded is type 11, code 0 meaning time to live exceeded in transit.

[3] Use the Traceroute Command on Operating Systems. Cisco.
https://www.cisco.com/c/en/us/support/docs/ip/ip-routed-protocols/22826-traceroute.html
Supports: Module 1 and 3: probe behavior on Cisco IOS, Linux and Windows. Cisco IOS and Linux traceroute send UDP probes starting at destination port 33434 with TTL 1 (default maximum 30 hops); hops return ICMP Time Exceeded and the destination returns ICMP Port Unreachable (type 3 code 3). Windows tracert uses ICMP Echo. Cisco routers rate-limit ICMP unreachables to one per 500 ms by default, which can show as asterisks for the final hop.

[4] mtr: Official repository for mtr, a network diagnostic tool. mtr project (GitHub).
https://github.com/traviscross/mtr
Supports: Module 1 and 3: continuous per-hop statistics and the meaning of intermediate-hop loss. mtr combines traceroute and ping, sends probes with increasing TTL and prints running per-hop statistics; routers may never send or may rate-limit ICMP responses, so apparent loss at an intermediate hop does not always mean forwarded traffic is lost there.

[5] Flowcharts Syntax | Mermaid. Mermaid (mermaid.js.org). https://mermaid.js.org/syntax/flowchart.html
Supports: Module 7: Mermaid flowchart syntax, subgraphs, shapes and classDef styling. Flowcharts start with flowchart (or graph) plus a direction (TB/TD, BT, RL, LR); links include -->, ---, dotted -.- and thick ===; subgraph id [Title] ... end groups nodes; from v11.3.0 shapes can be set with A@{ shape: ... }; classDef defines a reusable style applied with class; a lowercase end inside a node label breaks the diagram; commas in stroke-dasharray must be escaped.

[6] Creating diagrams - GitHub Docs. GitHub Docs. https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams
Supports: Module 7 and 9: rendering Mermaid in GitHub pull requests and Markdown files. GitHub renders Mermaid from a fenced code block labeled mermaid in issues, discussions, pull requests, wikis and Markdown files; a mermaid block containing info reveals which Mermaid version GitHub runs.

[7] Insert a diagram from specially formatted CSV data | draw.io. draw.io (diagrams.net). https://www.drawio.com/blog/insert-from-csv
Supports: Module 7: optional draw.io export for teams that maintain diagrams in draw.io. Arrange > Insert > Advanced > CSV takes # configuration lines before the CSV data; # label: uses %column% placeholders, # connect: takes a JSON object with from/to columns, # identity: lets a re-import update existing shapes instead of duplicating them, and # layout: auto applies an automatic layout.

[8] DOT Language | Graphviz. Graphviz. https://graphviz.org/doc/info/lang.html
Supports: Module 7: Graphviz DOT as an alternative renderer. DOT declares graph (edges --) or digraph (edges ->), optionally strict to forbid multi-edges; attributes are key=value pairs in brackets; graph/node/edge statements set defaults for objects defined afterwards; subgraphs named cluster* are drawn in a bounding box where the layout engine supports it.

[9] Prompting best practices. Anthropic (Claude Developer Platform docs). https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices
Supports: Module 4 and 7: prompting Claude for evidence-cited summaries and clean diagram output. Tell Claude what to do rather than what not to do, wrap instructions, context and input in XML tags to reduce misinterpretation, and use structured outputs or tool calling for fixed structure; examples are one of the most reliable ways to steer output format and structure.

[10] traceroute(8) - Linux manual page. man7.org (Linux man-pages). https://man7.org/linux/man-pages/man8/traceroute.8.html
Supports: Module 1 and 3: Linux traceroute options, methods and output annotations. Options include -I (ICMP ECHO probes), -T (TCP SYN probes, default destination port 80, intended to get past firewalls), -m max_ttl (default 30), -q nqueries (default 3 probes per hop), -w wait (default max 5.0 s), -N simultaneous probes (default 16; routers may throttle ICMP) and -n. The default method uses UDP to unlikely ports starting at 33434. No response prints *, and annotations such as !H, !N, !P and !X mark host, network, protocol unreachable and administratively prohibited.

[11] ping(8) - Linux manual page. man7.org (iputils). https://man7.org/linux/man-pages/man8/ping.8.html
Supports: Module 3: ping options and exit codes for deterministic wrappers. -c count stops after count ECHO_REQUESTs, -W timeout sets the per-reply wait in seconds, -w deadline sets an overall timeout; ping exits 1 if no reply is received (or fewer than count by the deadline), 2 on other errors, and 0 otherwise.

[12] tracert | Microsoft Learn. Microsoft. https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/tracert
Supports: Module 3: Windows tracert syntax and output for the Windows parser. Syntax tracert [/d] [/h <maximumhops>] [/j <hostlist>] [/w <timeout>] [/R] [/S <srcaddr>] [/4][/6] <targetname>; /d skips name resolution, /h default is 30 hops, /w default is 4000 ms and an asterisk is printed when no reply arrives in time; routers that do not return Time Exceeded show as a row of asterisks; pathping gives per-hop loss.

[13] RFC 4950: ICMP Extensions for Multiprotocol Label Switching. IETF / RFC Editor. https://www.rfc-editor.org/rfc/rfc4950
Supports: Module 1: MPLS hops and what traceroute can show inside an MPLS core. Defines an ICMP extension object that lets Label Switching Routers append the MPLS label stack to ICMP messages so enhanced traceroute can show MPLS information; an LSR that cannot deliver a labeled datagram removes the label stack and generates the ICMP message from the inner IP datagram.

[14] Detection, Understanding, and Prevention of Traceroute Measurement Artifacts. arXiv (Viger, Augustin et al.). https://www.arxiv.org/abs/0904.2733
Supports: Module 1 and 10: ECMP and load balancing artifacts in traceroute. Load balancing routers cause false loops, cycles and diamonds in traceroute output; Paris traceroute controls packet header contents so probes follow one path and gives a truer picture of the routes packets take.

[15] Configure permissions - Claude Code Docs. Anthropic (code.claude.com). https://code.claude.com/docs/en/permissions
Supports: Module 2 and 8: allow, ask and deny rules for Bash and MCP tools. Rules are evaluated deny, then ask, then allow, and the first match wins regardless of specificity; a bare tool name such as Bash in deny removes the

tool from context; mcp__<server>__* matches every tool from one server; Bash rules match command text and are not a security boundary around a program, so pair them with the sandbox when a restriction must hold; project rules live in .claude/settings.json.

[16] How Claude remembers your project - Claude Code Docs. Anthropic (code.claude.com). https://code.claude.com/docs/en/memory
Supports: Module 2: CLAUDE.md conventions for the pipeline repository. Project instructions live in ./CLAUDE.md or ./.claude/CLAUDE.md and are shared through version control; /init generates a starting CLAUDE.md; Claude treats CLAUDE.md as context, not enforced configuration, so use a PreToolUse hook to block an action.

[17] Connect Claude Code to tools via MCP - Claude Code Docs. Anthropic (code.claude.com). https://code.claude.com/docs/en/mcp
Supports: Module 2: registering the probe MCP server with Claude Code. Stdio servers are added with claude mcp add [options] <name> -- <command> [args...]; scopes are local (default), project (stored in .mcp.json at the project root and shared via version control) and user; interactive sessions ask for approval before using project-scoped servers.

[18] Give Claude custom tools - Claude Agent SDK. Anthropic (code.claude.com). https://code.claude.com/docs/en/agent-sdk/custom-tools
Supports: Module 2, 3 and 5: defining probe and device tools in the Python Agent SDK. Python tools use the @tool decorator (name, description, input schema, async handler) and are wrapped with create_sdk_mcp_server; tools are named mcp__{server}__{tool} and listed in allowed_tools; tools=[] removes all built-in tools so Claude can only use your MCP tools; a bare name in disallowed_tools removes a built-in from context; readOnlyHint annotations are metadata, not enforcement; handlers return content and optionally is_error.

[19] Get structured output from agents - Claude Agent SDK. Anthropic (code.claude.com). https://code.claude.com/docs/en/agent-sdk/structured-outputs
Supports: Module 4 and 10: schema-validated summaries and drift explanations. Pass output_format={'type': 'json_schema', 'schema': ...} (Pydantic model_json_schema() works) and read ResultMessage.structured_output; the SDK validates with JSON Schema draft-07 and re-prompts on mismatch; failure ends with subtype error_max_structured_output_retries, and a success result without structured_output must also be treated as a failure.

[20] Intercept and control agent behavior with hooks - Claude Agent SDK. Anthropic (code.claude.com). https://code.claude.com/docs/en/agent-sdk/hooks
Supports: Module 2 and 8: PreToolUse scope checks and PostToolUse audit logging. PreToolUse hooks registered with HookMatcher can return hookSpecificOutput.permissionDecision 'deny' with a permissionDecisionReason; matchers match tool names (MCP tools as mcp__<server>__<action>), not arguments, so check tool_input inside the hook; when several hooks disagree deny wins; PostToolUse suits audit logging; a timed-out PreToolUse hook blocks the call.

[21] Tools - Model Context Protocol specification (2025-06-18). Model Context Protocol. https://modelcontextprotocol.io/specification/2025-06-18/server/tools
Supports: Module 2 and 8: MCP tool discovery, results and security duties. Clients discover tools with tools/list and call them with tools/call; tools carry name, description, inputSchema, optional outputSchema and annotations (untrusted unless from trusted servers); results carry content, structuredContent and isError; servers MUST validate inputs, implement access controls, rate limit invocations and sanitize outputs; clients SHOULD keep a human able to deny invocations, use timeouts and log tool usage for audit.

[22] subprocess - Subprocess management. Python Software Foundation. https://docs.python.org/3/library/subprocess.html
Supports: Module 3: timeouts and exit-code handling in probe runners. With run(timeout=...), the child is killed and TimeoutExpired is raised after it terminates; CalledProcessError is raised for a non-zero exit when check=True and carries returncode, cmd and captured output.

[23] JSON Schema | Pydantic Docs. Pydantic. https://docs.pydantic.dev/latest/concepts/json_schema/
Supports: Module 4: generating the path record schema. BaseModel.model_json_schema() returns a jsonable dict of the model's JSON Schema; Pydantic targets JSON Schema Draft 2020-12 and OpenAPI 3.1.0.

[24] Models | Pydantic Docs. Pydantic. https://docs.pydantic.dev/latest/concepts/models/
Supports: Module 3 and 4: validating parsed probe output. model_validate() validates an object and model_validate_json() validates a JSON string or bytes; ValidationError is raised when data cannot be parsed into a model instance.

[25] APIRef_tm_ltm_virtual. F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_ltm_virtual.html
Supports: Module 5: reading F5 virtual servers. The collection /mgmt/tm/ltm/virtual supports OPTIONS and GET; properties include destination (virtual address and port), pool (default pool), ipProtocol and the partition (tmPartition, default Common).

[26] APIRef_tm_ltm_pool_members. F5 (clouddocs.f5.com). https://clouddocs.f5.com/api/icontrol-rest/APIRef_tm_ltm_pool_members.html
Supports: Module 5: reading F5 pool members and their state. Pool members live at /mgmt/tm/ltm/pool/~<pool id>/members; properties include address, state (user-down forces the member offline), session (default user-enabled) and monitor.

[27] Demystifying iControl REST Part 3 - How to pass query parameters and tmsh options. F5 DevCentral. https://community.f5.com/t5/technical-articles/demystifying-icontrol-rest-part-3-how-to-pass-query-parameters/ta-p/286957
Supports: Module 5: efficient read-only F5 queries. iControl REST supports OData $filter, $select, $skip and $top, plus expandSubcollections (returns subcollections such as pool members in the first request), options (adds tmsh arguments) and ver (pins behavior to a TMOS version); example /mgmt/tm/ltm/pool?$filter=partition+eq+staging&$select=name,loadBalancingMode.

[28] lbvserver | configuration (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/lb/lbvserver.html
Supports: Module 5: reading NetScaler load balancing virtual servers. GET (all) is /nitro/v1/config/lbvserver with attrs and filter query parameters; fields include name, servicetype, ipv46, port and read-only curstate and effectivestate (UP, DOWN, OUT OF SERVICE and others); requests carry Cookie: NITRO_AUTH_TOKEN, or X-NITRO-USER and X-NITRO-PASS headers for single authenticated operations.

[29] lbvserver_servicegroup_binding | configuration (NITRO API). NetScaler (developer-docs.netscaler.com). https://developer-docs.netscaler.com/en-us/adc-nitro-api/current-release/configuration/lb/lbvserver_servicegroup_binding.html
Supports: Module 5: mapping a NetScaler vserver to its service groups. The binding object /nitro/v1/config/lbvserver_servicegroup_binding links a virtual server name to servicegroupname (with weight and order); add is a PUT and returns 201 Created.

[30] alteon-sdk (PyPI project metadata). PyPI (package published by a Radware author). https://pypi.org/pypi/alteon-sdk/json
Supports: Module 5: Radware Alteon access from Python. alteon-sdk provides a Python API to Alteon structures via a REST backend, with beans such as SlbNewCfgEnhRealServerTable read through AlteonClient(...).api.device.read(bean); it requires Python 3.6 or later and lists minimum supported Alteon versions 31.0.10.0 and 32.2.2.0; the latest release on PyPI is a beta (0.16b2).

[31] Cisco APIC REST API Configuration Guide, Release 4.2(x) and Later - Using the REST API. Cisco. https://www.cisco.com/c/en/us/td/docs/dcn/aci/apic/all/apic-rest-api-configuration-guide/cisco-apic-rest-api-configuration-guide-42x-and-later/m_using_the_rest_api.html
Supports: Module 6: APIC login, class and managed object queries. Log in with POST https://<apic>/api/aaaLogin.json and an aaaUser object; the token is sent back as the APIC-cookie; class queries use /api/class/<className>.json and MO queries /api/mo/<dn>.json; query-target is self, children or subtree (default self), query-target-filter takes expressions such as eq(attribute,value), and rsp-subtree is no, children or full (default no).

[32] Cisco Live LTRDCN-3225 ACI programmability lab: class queries and children. Cisco Live. https://aci-prog-lab.ciscolive.com/lab/pod1/js/js-app2
Supports: Module 6: locating an endpoint on a leaf. The children of fvCEp hold the endpoint's location in the

fabric, specifically fvRsCEpToPathEp; query /api/node/class/fvCEp.json?rsp-subtree=children&rsp-subtree-class=fvRsCEpToPathEp style requests to include them, because a plain fvCEp query does not return its children.

[33] ACI Sandboxes - Cisco DevNet. Cisco DevNet. https://developer.cisco.com/docs/aci/sandbox
Supports: Lab environment: free APIC access. DevNet offers an Always-On ACI Simulator (shared, no reservation) and a reservable ACI Simulator sandbox (up to 5 consecutive days) for REST API work over HTTP/HTTPS with XML and JSON.

[34] Configure route tables - Amazon VPC. Amazon Web Services.
https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html
Supports: Module 6: where traceroute stops and VPC route tables begin. A route table holds routes that decide where traffic from a subnet or gateway is directed; each route has a destination (CIDR or prefix list) and a target such as an internet gateway, NAT gateway, peering connection or VPN connection; a main route table is created with every VPC.

[35] Next hop overview - Azure Network Watcher. Microsoft. https://learn.microsoft.com/en-us/azure/network-watcher/next-hop-overview
Supports: Module 6: Azure next hop as cloud path evidence. Next hop returns the next hop type, IP address and route table ID for a destination; types include Internet, VirtualAppliance, VirtualNetworkGateway, VirtualNetwork, VirtualNetworkPeering, VirtualNetworkServiceEndpoint, MicrosoftEdge and None; System Route is returned when no user-defined route applies.

[36] Usage | Mermaid. Mermaid (mermaid.js.org). https://mermaid.js.org/config/usage.html
Supports: Module 7: validating Mermaid without rendering, and security level. mermaid.parse(text, parseOptions) checks a definition without rendering; it returns an object with diagramType when valid and throws (or returns false with suppressErrors) when invalid; securityLevel strict (default) encodes HTML tags and disables click functionality.

[37] mermaid-cli. Mermaid project (GitHub). https://github.com/mermaid-js/mermaid-cli
Supports: Module 7: deterministic command-line rendering. Install with npm install -g @mermaid-js/mermaid-cli and run mmdc -i input.mmd -o output.svg to produce SVG, PNG or PDF; a Docker image is published as minlag/mermaid-cli.

[38] Sequence diagrams | Mermaid. Mermaid (mermaid.js.org).
https://mermaid.js.org/syntax/sequenceDiagram.html
Supports: Module 7: request-response flow diagrams. sequenceDiagram declares participant and actor lines (with aliases), solid ->> and dashed -->> messages, activate/deactivate, autonumber and Note over A,B for annotations.

[39] Table API (Zurich release). ServiceNow.
https://www.servicenow.com/docs/r/MVRCUFVyKWd7vUoUukbW6g/slehVLrQON4zvkIDTxVDQg
Supports: Module 9: updating the requesting record. Endpoints are GET/POST /now/table/{tableName} and GET/PUT/PATCH/DELETE /now/table/{tableName}/{sys_id}; sysparm_query is an encoded query, sysparm_fields limits returned fields, sysparm_limit caps records (default 10000).

[40] Attachment API (Xanadu release). ServiceNow.
https://www.servicenow.com/docs/r/zZBaGIzc7EPkA0ySnMn5eg/6cG37dgmR1vwJCbI7a6lCg
Supports: Module 9: attaching the diagram and evidence bundle. POST /now/attachment/file uploads a binary file with required query parameters table_name, table_sys_id and file_name; POST /now/attachment/upload takes multipart form data; GET /now/attachment/{sys_id}/file returns the file.

[41] REST step (Workflow Studio, Brazil release). ServiceNow. https://www.servicenow.com/docs/r/build-workflows/workflow-studio/rest-request-action-designer.html
Supports: Module 9: Flow Designer calling the pipeline. The REST step sends an outbound REST request to an external system; it is not in the base system and requires an Integration Hub subscription; it supports a Connection Alias and an option to run through a MID Server.

[42] REST API endpoints for pull requests. GitHub Docs. https://docs.github.com/en/rest/pulls/pulls
Supports: Module 9 and 10: opening the diagram pull request. POST /repos/{owner}/{repo}/pulls takes title, head

(the branch with changes), base (the target branch), body and draft; it returns 201 Created, 403 or 422.

[43] Events that trigger workflows - schedule. GitHub Docs. https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows
Supports: Module 10: scheduled re-mapping. on: schedule uses five-field POSIX cron in UTC by default (an IANA timezone can be set); runs use the latest commit on the default branch; the shortest interval is every 5 minutes; runs can be delayed or dropped at high load, especially at the start of every hour; in public repositories scheduled workflows are disabled after 60 days without activity; @daily-style shortcuts are not supported.

[44] GITHUB_TOKEN - GitHub Docs. GitHub Docs. https://docs.github.com/en/actions/concepts/security/github_token
Supports: Module 10: what happens when the drift job opens a pull request. GITHUB_TOKEN is a GitHub App installation token limited to the repository; events it triggers do not create new workflow runs, except workflow_dispatch and repository_dispatch, and pull_request opened, synchronize or reopened events, which create runs in an approval-required state that a user with write access must approve.

[45] BIG-IP Virtual Edition free trial. F5. https://www.f5.com/trials/big-ip-virtual-edition
Supports: Lab environment: F5 BIG-IP VE for labs. F5 offers a 30-day free trial of BIG-IP Virtual Edition and BIG-IQ Virtual Edition; trial keys are requested through a MyF5 account under Trials.

[46] NetScaler VPX FAQ (13.1). NetScaler (docs.netscaler.com). https://docs.netscaler.com/en-us/vpx/13-1/deploy-vpx-faq.html
Supports: Lab environment: NetScaler VPX Express for labs. VPX Express is free, needs no license file and has no expiry in the newer version (older Express licenses expire one year after download); it has the Standard Edition feature set except Gateway from 12.0-56.20; a retail license is needed to file support cases.

For more info - https://gennoor.com

Gennoor Tech TRAIN. INNOVATE. TRANSFORM. For more info - https://gennoor.com